

Asynchronous FIFO

1. What a FIFO Is (Hardware Meaning)

A FIFO (First-In First-Out) is a **hardware buffer** that stores data so that:

- data is read in the same order it was written
- producer and consumer are decoupled in time

A FIFO is **not** a list or queue abstraction.

It is a **memory + control logic**.

At minimum, a FIFO contains:

- a memory array
 - a write pointer
 - a read pointer
 - logic to prevent overflow and underflow
-

2. Why FIFOs Exist

FIFOs are used when:

- producer and consumer run at different speeds
- producer and consumer use different clocks
- data arrives in bursts
- flow control is required

An **asynchronous FIFO** specifically solves:

| communication between two unrelated clock domains

3. Synchronous vs Asynchronous FIFO

Synchronous FIFO

- one clock
- read and write use same clock
- no CDC issues
- simpler design

Asynchronous FIFO

- two clocks
 - write clock (tx_clk)
 - read clock (rx_clk)
- clocks have no phase or frequency relationship
- requires CDC-safe design
- uses Gray code + synchronizers

This assignment is about an **asynchronous FIFO**.

4. FIFO Memory Organization

The FIFO uses a **circular buffer (ring buffer)**.

```
mem[0], mem[1], mem[2], ..., mem[DEPTH-1]
```

When the end is reached, indexing wraps back to 0.

5. What “Pointer” Means in Hardware

A FIFO pointer is **not a C pointer**.

A pointer in FIFO design is:

| a counter that holds the index of the next memory location

Write Pointer

- indicates where the next write will occur
- increments on each successful write

Read Pointer

- indicates where the next read will occur
- increments on each successful read

Pointers are **state**, stored in registers.

6. Pointer Width (Critical Detail)

If FIFO depth = `DEPTH`

Address bits needed = `log2(DEPTH)`

Pointer width = `log2(DEPTH) + 1`

Why the extra bit?

- lower bits → memory index
- MSB → tracks wrap-around history

This extra bit allows differentiation between:

- **empty**
 - **full**
-

7. Empty and Full Conditions

Let:

- `wr_ptr` = write pointer
- `rd_ptr` = read pointer

Empty

`wr_ptr == rd_ptr`

Full

`lower bits equal MSB different`

This works only because pointers are one bit wider than address width.

8. Clock Domains and Ownership Rule

The FIFO does **not** have a single clock.

It has:

- a **write clock domain**
- a **read clock domain**

Ownership rule (very important)

| Each clock domain owns and updates only its own pointer.

- write pointer:
 - updated only on write clock
- read pointer:
 - updated only on read clock

No clock domain ever updates or trusts a foreign pointer directly.

9. Why Pointers Must Be Synchronized

The write side must know:

- where the **read pointer** is (to detect full)

The read side must know:

- where the **write pointer** is (to detect empty)

But:

- pointers live in different clock domains
- direct sampling causes metastability

Therefore:

- pointers must be synchronized before use
-

10. Pointer Synchronization Strategy

What crosses clock domains

- **Only pointers**
- Never data
- Never valid/ready/credit

How pointers cross

1. Convert binary pointer → Gray code
 2. Pass through a 2-flip-flop synchronizer
 3. Use synchronized Gray pointer for status logic
-

11. Why Gray Code Is Used

Binary counters can change multiple bits at once:

0111 → 1000

If sampled mid-transition, illegal values can appear.

Gray code guarantees:

| only one bit changes between successive values

This makes cross-clock sampling safe.

12. Two Versions of Each Pointer (Key Insight)

Each pointer exists in **two forms**.

Write Pointer

- **Authoritative version**
 - lives in write clock domain
 - used for writing memory
- **Synchronized copy**
 - lives in read clock domain
 - used only for empty detection

Read Pointer

- **Authoritative version**
 - lives in read clock domain
 - used for reading memory
- **Synchronized copy**
 - lives in write clock domain
 - used only for full detection

A domain **never uses the synchronized copy of its own pointer**.

13. Why the Synchronization Direction Matters

- Read pointer → synchronized into **write domain**
 - so write side can detect FULL
- Write pointer → synchronized into **read domain**
 - so read side can detect EMPTY

This is bidirectional crossing, but **each pointer crosses only one way**.

14. Metastability (Why This Is Necessary)

When a signal is sampled while changing:

- flip-flop output can become undefined
- resolution time is unbounded

This is metastability.

Async FIFO design avoids failure by:

- synchronizing pointers
 - tolerating delayed information
 - designing for safety, not immediacy
-

15. Valid-Ready Interface (Write / TX Side)

Signals:

- `tx_valid` — producer has data
- `tx_ready` — FIFO can accept data

Write occurs when:

```
tx_valid && tx_ready
```

`tx_ready` is asserted when:

```
FIFO is not full
```

16. Valid-Credit Interface (Read / RX Side)

Signals:

- `rx_valid` — FIFO has data
- `rx_credit` — receiver grants permission to send data

Credit represents **capacity in advance**.

Rules:

- FIFO may send data only if:
 - FIFO is not empty
 - credit count > 0
- each data transfer consumes one credit

This is flow control without backpressure.

17. Why Credit ≠ Ready

- Ready is reactive (backpressure)
- Credit is proactive (budget-based)

Credits are common in:

- NoCs
 - high-performance interconnects
 - latency-sensitive systems
-

18. Memory Behavior in Async FIFO

The memory is effectively **dual-port**:

- write port uses write clock
- read port uses read clock

No data signal crosses clock domains.

19. What Correctness Means in an Async FIFO

The FIFO must guarantee:

- no overwrite of unread data
- no read of unwritten data
- preserved ordering

It does **not** require:

- zero latency
- instantaneous knowledge of other domain

Safety > speed.

20. One-Sentence Mental Model

An asynchronous FIFO is two independent state machines sharing memory, communicating progress via synchronized pointers.

If you understand that sentence, you understand async FIFOs.

21. Why This Topic Feels Deep

Because:

- clocks are not absolute
- logic must tolerate uncertainty
- correctness is ensured probabilistically

Async FIFOs sit at the boundary between:

- digital abstraction
- physical reality

That's why they are taught — and feared.